Министерство образования Республики Беларусь
Учреждение Образования
«Белорусский государственный университет информатики и радиоэлектроники»

Факультет компьютерных систем и сетей

Кафедра электронных вычислительных машин

Лабораторная работа № 1
по дисциплине «Жизненный цикл разработки программного обеспечения»

Выполнил
студент гр. 439401
Чурик М.В.

Проверил
Силич С.С.

Минск 2026

Цель работы: разработать утилиту для информирования пользователя о нормальном артериальном давлении. На главной форме поместить кнопку «Ввести данные». В отдельном окне запросить у пользователя дату рождения (не используя стандартный элемент ввода даты), вес, текущее давление. Последние введенные данные нужно запоминать в главной форме и восстанавливать при повторном вводе. После этого на главной форме отобразить результат – «идеальное давление» для возраста и веса, оценку текущего давления человека (повышенное, пониженное, норма). Если введены данные – показать сообщение об ошибке.

Теоретические сведения

Model-View-Controller (MVC) – это фундаментальный шаблон проектирования, предназначенный для разделения бизнес-логики приложения (компонентов работы с данными) от его визуального представления.

Согласно классической концепции, архитектура приложения разделяется на три изолированных класса:

- Модель (Model) – управляет внутренним состоянием, данными и бизнес-логикой домена приложения. Она отвечает на запросы информации о своем состоянии и обрабатывает инструкции по его изменению. Модель является полностью независимым компонентом: она ничего не знает ни о Представлении, ни о Контроллере.
- Представление (View) – отвечает за графический интерфейс (GUI), визуальное отображение данных пользователю и презентационную логику.
- Контроллер (Controller) – перехватывает действия пользователя (нажатия клавиш, клики мыши), интерпретирует их и вызывает соответствующие методы Модели для изменения её состояния.

Активная и пассивная модели в MVC

В зависимости от способа синхронизации данных и интерфейса, автор паттерна Стив Бурбек (Steve Burbeck) выделяет две ключевые модификации MVC:

- Пассивная модель (Passive Model):

Используется в архитектурах, где Контроллер является единственным источником изменений Модели. В этом сценарии Модель полностью автономна и не способна сообщать о смене своего состояния. Контроллер сначала модифицирует Модель, а затем самостоятельно и принудительно дает команду Представлению обновить отображение на экране. Классическим примером такой схемы является стандартный протокол HTTP в веб-приложениях.

- Активная модель (Active Model):

Применяется, когда состояние Модели может меняться без прямого участия текущего Контроллера (например, при поступлении внешних данных, вычислениях по таймеру или многопоточной обработке). Так как только сама Модель способна зафиксировать изменения своего внутреннего состояния,

она обязана уведомить интерфейс о необходимости синхронизации. Чтобы сохранить независимость Модели от конкретных окон интерфейса, активная модель реализуется через паттерн Observer («Наблюдатель»), также известный как схема *«издатель-подписчик» (publish-subscribe)*. Представления регистрируются в Модели в качестве наблюдателей. При изменении бизнес-данных Модель циклически обходит список подписчиков и автоматически генерирует для них сигналы оповещения, изолируя логику расчета от визуального кода.

Современная IDE (на примере связки VS Code и инструментария GCC/MinGW) — это комплекс программных средств для автоматизации разработки, состоящий из следующих компонентов:

- Текстовый редактор кода — оболочка с функциями подсветки синтаксиса и структурирования исходных файлов.
- Компилятор и Линкер (g++.exe) — транслятор кода C++ в машинные объектные модули и компоновщик их с системными библиотеками в файл .exe.
- Система автоматизации сборки (tasks.json) — диспетчер, управляющий последовательностью компиляции и передачей флагов оптимизации.
- Отладчик (gdb.exe) — инструмент для пошагового выполнения программы, установки точек останова (breakpoints) и инспекции переменных в памяти.
- Менеджер расширений — подсистема интеграции плагинов (например, для поддержки UML и SysMLv2).

Основные приемы работы с редактором кода

Эффективное написание кода и снижение числа ошибок обеспечиваются следующими встроенными технологиями редактора:

- Автодополнение (IntelliSense) — контекстные подсказки имен функций, классов и типов данных на лету при вводе первых букв.
- Подсветка синтаксиса — цветовое выделение ключевых слов языка, типов данных, строк и комментариев для улучшения читаемости кода.
- Фоновый статический анализ (Линтинг) — мгновенное обнаружение и подчеркивание синтаксических ошибок (например, несовместимости типов в WinAPI) еще до запуска компиляции.
- Навигация и рефакторинг — быстрое перемещение к объявлению функций (клавиша F12) и безопасное массовое переименование переменных по всему проекту.

Код программы

```
#include <windows.h>
#include <string>
#include <cmath>
#include <stdexcept>

using namespace std;

// ========================================
// 1. МОДЕЛЬ (Model) - Бизнес-логика
// ========================================
class BloodPressureModel {
private:
    int birthDay = 0, birthMonth = 0, birthYear = 0;
    double weight = 0.0;
    int currentSys = 0, currentDia = 0;

    int idealSys = 0, idealDia = 0;
    wstring assessment = L"-";

public:
    HWND viewWindow = nullptr;

    void updateData(int d, int m, int y, double w, int sys, int dia) {
        if (d < 1 || d > 31 || m < 1 || m > 12 || y < 1900 || y > 2026
|| w <= 0 || sys <= 0 || dia <= 0) {
            throw invalid_argument("Введены некорректные данные!");
        }

        birthDay = d; birthMonth = m; birthYear = y;
        weight = w; currentSys = sys; currentDia = dia;

        int age = 2026 - birthYear;
        idealSys = static_cast<int>(round(109 + (0.5 * age) + (0.1 *
weight)));
        idealDia = static_cast<int>(round(63 + (0.1 * age) + (0.15 *
weight)));

        int sysDiff = currentSys - idealSys;
        int diaDiff = currentDia - idealDia;

        if (sysDiff > 15 || diaDiff > 10) assessment = L"Повышенное";
        else if (sysDiff < -15 || diaDiff < -10) assessment =
L"Пониженное";
        else assessment = L"Норма";

        if (viewWindow) {
            PostMessageW(viewWindow, WM_USER + 1, 0, 0);
        }
    }

    int getBirthDay() const { return birthDay; }
    int getBirthMonth() const { return birthMonth; }
    int getBirthYear() const { return birthYear; }
    double getWeight() const { return weight; }
    int getCurrentSys() const { return currentSys; }
    int getCurrentDia() const { return currentDia; }

    int getIdealSys() const { return idealSys; }
    int getIdealDia() const { return idealDia; }
    wstring getAssessment() const { return assessment; }
};
```

```
BloodPressureModel g_model;

// ========================================
// 2. КОНТРОЛЛЕР (Controller)
// ========================================
class BloodPressureController {
public:
    bool trySaveData(const wstring& dStr, const wstring& mStr, const
wstring& yStr,
                     const wstring& wStr, const wstring& sysStr, const
wstring& diaStr,
                     wstring& outError) {
        try {
            int d = stoi(dStr);
            int m = stoi(mStr);
            int y = stoi(yStr);
            double w = stod(wStr);
            int sys = stoi(sysStr);
            int dia = stoi(diaStr);

            g_model.updateData(d, m, y, w, sys, dia);
            return true;
        }
        catch (...) {
            outError = L"Ошибка ввода! Проверьте корректность чисел.";
            return false;
        }
    }
};

BloodPressureController g_controller;

HWND hDay, hMonth, hYear, hWeight, hSys, hDia;
HWND hLblIdeal, hLblCurrent, hLblAssessment;

// ========================================
// 3. ПРЕДСТАВЛЕНИЕ (View) - Интерфейс
// ========================================

LRESULT CALLBACK InputWndProc(HWND hwnd, UINT msg, WPARAM wp, LPARAM lp)
{
    switch (msg) {
    case WM_CREATE: {
        CreateWindowW(L"Static", L"Дата (ДД ММ ГГГГ):", WS_VISIBLE |
WS_CHILD, 10, 10, 150, 20, hwnd, NULL, NULL, NULL);
        hDay = CreateWindowW(L"Edit", L"", WS_VISIBLE | WS_CHILD |
WS_BORDER, 160, 10, 30, 20, hwnd, NULL, NULL, NULL);
        hMonth = CreateWindowW(L"Edit", L"", WS_VISIBLE | WS_CHILD |
WS_BORDER, 200, 10, 30, 20, hwnd, NULL, NULL, NULL);
        hYear = CreateWindowW(L"Edit", L"", WS_VISIBLE | WS_CHILD |
WS_BORDER, 240, 10, 50, 20, hwnd, NULL, NULL, NULL);

        CreateWindowW(L"Static", L"Вес (кг):", WS_VISIBLE | WS_CHILD,
10, 40, 150, 20, hwnd, NULL, NULL, NULL);
        hWeight = CreateWindowW(L"Edit", L"", WS_VISIBLE | WS_CHILD |
WS_BORDER, 160, 40, 130, 20, hwnd, NULL, NULL, NULL);

        CreateWindowW(L"Static", L"Верхнее (систол.):", WS_VISIBLE |
WS_CHILD, 10, 70, 150, 20, hwnd, NULL, NULL, NULL);
        hSys = CreateWindowW(L"Edit", L"", WS_VISIBLE | WS_CHILD |
WS_BORDER, 160, 70, 130, 20, hwnd, NULL, NULL, NULL);
```

```
            CreateWindowW(L"Static", L"Нижнее (диастол.):", WS_VISIBLE |
WS_CHILD, 10, 100, 150, 20, hwnd, NULL, NULL, NULL);
            hDia = CreateWindowW(L"Edit", L"", WS_VISIBLE | WS_CHILD |
WS_BORDER, 160, 100, 130, 20, hwnd, NULL, NULL, NULL);

            if (g_model.getBirthYear() != 0) {
                SetWindowTextW(hDay,
to_wstring(g_model.getBirthDay()).c_str());
                SetWindowTextW(hMonth,
to_wstring(g_model.getBirthMonth()).c_str());
                SetWindowTextW(hYear,
to_wstring(g_model.getBirthYear()).c_str());
                SetWindowTextW(hWeight,
to_wstring(static_cast<int>(g_model.getWeight())).c_str());
                SetWindowTextW(hSys,
to_wstring(g_model.getCurrentSys()).c_str());
                SetWindowTextW(hDia,
to_wstring(g_model.getCurrentDia()).c_str());
            }

            CreateWindowW(L"Button", L"Сохранить", WS_VISIBLE | WS_CHILD,
100, 140, 100, 30, hwnd, (HMENU)1, NULL, NULL);
            break;
        }
        case WM_COMMAND: {
            if (LOWORD(wp) == 1) {
                // ИСПРАВЛЕНО: Выделены корректные массивы-буферы для строк
                wchar_t d[10], m[10], y[10], w[20], sys[20], dia[20];
                GetWindowTextW(hDay, d, 10);
                GetWindowTextW(hMonth, m, 10);
                GetWindowTextW(hYear, y, 10);
                GetWindowTextW(hWeight, w, 20);
                GetWindowTextW(hSys, sys, 20);
                GetWindowTextW(hDia, dia, 20);

                wstring error;
                if (g_controller.trySaveData(d, m, y, w, sys, dia, error)) {
                    DestroyWindow(hwnd);
                } else {
                    MessageBoxW(hwnd, error.c_str(), L"Ошибка", MB_OK |
MB_ICONERROR);
                }
            }
            break;
        }
        default:
            return DefWindowProcW(hwnd, msg, wp, lp);
        }
        return 0;
    }

    LRESULT CALLBACK MainWndProc(HWND hwnd, UINT msg, WPARAM wp, LPARAM lp)
{   switch (msg) {
        case WM_CREATE: {
            g_model.viewWindow = hwnd;

            CreateWindowW(L"Button", L"Ввести данные", WS_VISIBLE |
WS_CHILD, 20, 20, 150, 30, hwnd, (HMENU)1, NULL, NULL);
            hLblIdeal = CreateWindowW(L"Static", L"Идеальное давление: Не
рассчитано", WS_VISIBLE | WS_CHILD, 20, 70, 350, 20, hwnd, NULL, NULL, NULL);
            hLblCurrent = CreateWindowW(L"Static", L"Введенное давление: -",
WS_VISIBLE | WS_CHILD, 20, 100, 350, 20, hwnd, NULL, NULL, NULL);
            hLblAssessment = CreateWindowW(L"Static", L"Оценка состояния: -
", WS_VISIBLE | WS_CHILD, 20, 130, 350, 20, hwnd, NULL, NULL, NULL);
```

```
            break;}
        case WM_COMMAND: {
            if (LOWORD(wp) == 1) {
                CreateWindowExW(0, L"InputWindowClass", L"Input Data",
WS_OVERLAPPEDWINDOW | WS_VISIBLE,
                    CW_USEDEFAULT, CW_USEDEFAULT, 330, 220, hwnd, NULL,
NULL, NULL);}
            break;}
        case WM_USER + 1: {
            wstring idealText = L"Идеальное давление: " +
to_wstring(g_model.getIdealSys()) + L" / " +
to_wstring(g_model.getIdealDia()) + L" мм рт. ст.";
            wstring currentText = L"Введенное давление: " +
to_wstring(g_model.getCurrentSys()) + L" / " +
to_wstring(g_model.getCurrentDia()) + L" мм рт. ст.";
            wstring assessText = L"Оценка состояния: " +
g_model.getAssessment();

            SetWindowTextW(hLblIdeal, idealText.c_str());
            SetWindowTextW(hLblCurrent, currentText.c_str());
            SetWindowTextW(hLblAssessment, assessText.c_str());
            break;
        }
        case WM_DESTROY:
            PostQuitMessage(0);
            break;
        default:
            return DefWindowProcW(hwnd, msg, wp, lp);
        }
        return 0;
    }

    int WINAPI WinMain(HINSTANCE hInst, HINSTANCE, LPSTR, int nCmdShow) {
        WNDCLASSW wcMain = {0};
        wcMain.lpfnWndProc = MainWndProc;
        wcMain.hInstance = hInst;
        wcMain.hbrBackground = (HBRUSH)(COLOR_WINDOW + 1);
        wcMain.lpszClassName = L"MainWindowClass";
        RegisterClassW(&wcMain);

        WNDCLASSW wcInput = {0};
        wcInput.lpfnWndProc = InputWndProc;
        wcInput.hInstance = hInst;
        wcInput.hbrBackground = (HBRUSH)(COLOR_WINDOW + 1);
        wcInput.lpszClassName = L"InputWindowClass";
        RegisterClassW(&wcInput);

        HWND hwnd = CreateWindowExW(
            0,
            L"MainWindowClass",
            L"BloodPressureAPP",
            WS_OVERLAPPEDWINDOW | WS_VISIBLE,
            CW_USEDEFAULT, CW_USEDEFAULT, 400, 230,
            NULL, NULL, hInst, NULL
        );

        MSG msg = {0};
        while (GetMessageW(&msg, NULL, 0, 0)) {
            TranslateMessage(&msg);
            DispatchMessageW(&msg);
        }
        return 0;
    }
```

Вывод программы



Рисунок 1 – Главный экран

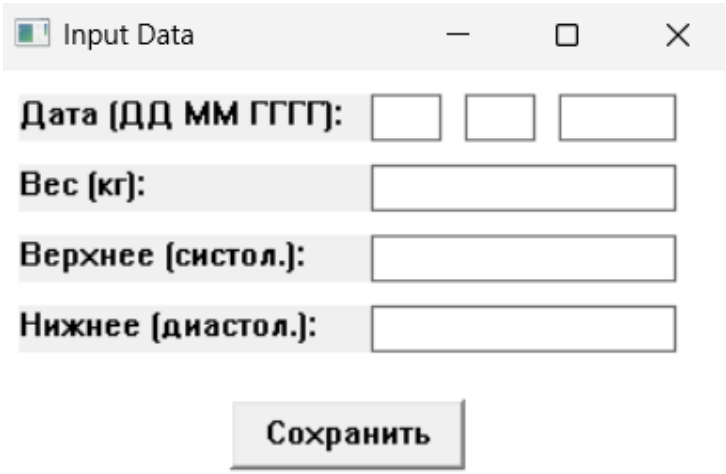


Рисунок 2 – Меню для ввода данных

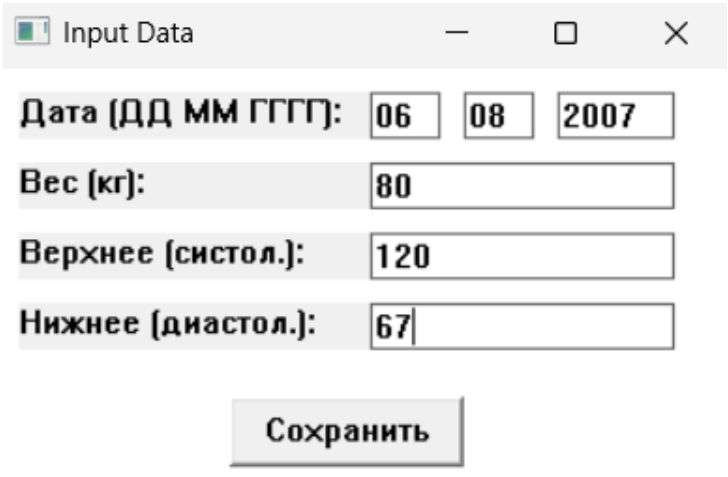


Рисунок 3 – Меню после ввода данных

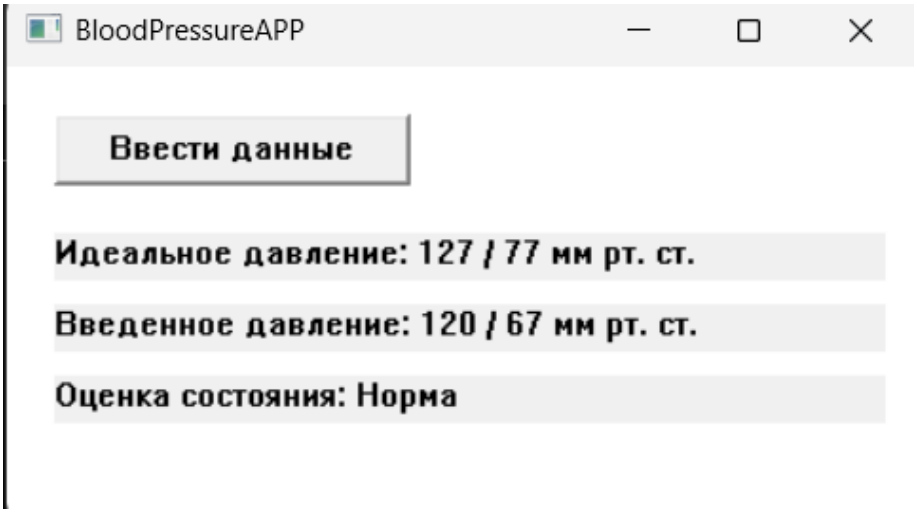


Рисунок 4 – Главный экран после заполнения данными

Заключение

В ходе выполнения лабораторной работы было разработано графическое приложение, полностью отвечающее требованиям задания. Реализованы функции расчета идеального давления по формуле Волынского на основе даты рождения и веса, оценка состояния пользователя, а также механизм восстановления ранее введенных данных.

Приложение спроектировано по паттерну MVC с активной моделью: синхронизация данных и интерфейса реализована через системные сообщения на основе паттерна «Наблюдатель». Использование низкоуровневого Windows API для построения оконного интерфейса позволило полностью выполнить требование об отказе от высокоуровневых фреймворков. Разработка, сборка автономного .exe файла и пошаговая отладка проекта успешно выполнены в среде MS Visual Studio Code с компилятором MinGW.